

GradeMate

Project Team

Amna Khan 21I-0345
Shanzay Alam 21I-2707
Ansab Alam 21I-2706

Session 2021-2025

Supervised by

Ms. Saira Qamar



Department of Artificial Intelligence

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	1
1.3	Modules	2
1.3.1	Collect Dataset	2
1.3.2	Model training	2
1.3.3	Quiz Evaluation	3
1.3.4	Automated Grading	3
1.4	User Classes and Characteristics	3
2	Project Requirements	5
2.1	Use-case Diagram	5
2.2	Functional Requirements	11
2.2.1	Module 1: Collecting Dataset	11
2.2.2	Module 2: Model Training	11
2.2.3	Module 3: Quiz Evaluation	11
2.2.4	Module 4: Automated Grading	12
2.3	Non-Functional Requirements	12
2.3.1	Reliability	12
2.3.2	Usability	13
2.3.3	Performance	13
2.3.4	Security	13
2.4	Domain Model	14
3	System Overview	15
3.1	Architectural Design	15
3.2	Design Models	16
3.3	Data Design	25
3.3.1	Transformation of Information Domain into Data Structures	25
3.3.2	Storage of Major Data Entities	26
3.3.3	Processing of Major Data Entities	26
3.3.4	Organization of Data Entities	27

3.3.5	Databases and Data Storage Items	27
4		29
4.1	Implementation and Testing	29
4.1.1	Functionality, Context, and Design	29
4.1.1.1	Functionality	29
4.1.1.2	Context.....	30
4.1.1.3	Design	30
4.1.1.4	Background Information	30
4.2	Algorithm Design	31
4.3	External APIs/SDKs	33
4.4	Testing Details	33
4.4.1	Unit Testing	33
5	Conclusions and Future Work	35
5.1	Conclusion.....	35
5.2	Future Work.....	35
	References	37

List of Figures

2.1	Use Case Diagram	5
2.2	Domain Model.....	14
3.1	The architectural design of the GradeMate system.....	15
3.2	Data collection Module	16
3.3	Model Training Module.....	17
3.4	Automated Grading Module	18
3.5	Quiz Evaluation Module.....	19
3.6	OCR Conversion and Quiz Scanning.....	20
3.7	Model Training.....	21
3.8	Quiz Evaluation	22
3.9	Automated Grading	23
3.10	State Transition Diagram	24
3.11	Class Diagram	25
4.1	Pseudocode for OCR-based Text Extraction.....	31
4.2	Pseudocode for rubric-based quiz evaluation	32
4.3	Pseudocode for Automatic Grading.....	32
4.4	Pseudocode for Plagiarism Detection.	32

List of Tables

2.1	Detailed Use Case: Manage Handwritten Quizzes	6
2.2	Detailed Use Case: Manage Grading Criteria.....	7
2.3	Detailed Use Case: View Graded Quizzes.....	8
2.4	Detailed Use Case: Plagiarism Detection	9
2.5	Detailed Use Case: Dataset Preprocessing and Cleaning	10
4.1	Unit Test Case for quiz upload.	33
4.2	Unit Test Case for OCR conversion.....	33
4.3	Unit Test Case for Quiz Evaluation	34
4.4	Unit Test Case for model training.....	34
4.5	Unit Test Case for grading criteria.....	34
4.6	Unit Test Case for plagiarism detection.....	34

Chapter 1

Introduction

The purpose of this project proposal is to provide a comprehensive overview of the development of GradeMate, which is an application designed to automate the grading process of handwritten quizzes written in C++ programming language. This document will outline the major aspects of the project including the objective, scope, and high-level goals of the system.

1.1 Problem Statement

The evaluation of handwritten programming quizzes in C++ can be a very time-consuming and error-prone process. Commonly practiced traditional manual grading methods can result in variability and vagueness, which can lead to unfair evaluation of student work. Additionally, teachers spend a high amount of time collecting, checking and grading each quiz submission, leaving less time for interactive teaching and focus for the students.[1]. One common issue is that a student attempts a quiz and provides a correct solution but uses a different approach than the one stated or expected by the teacher. Now the teacher, who has to manually check the quizzes of the entire class, struggles to assess this alternate solution.

1.2 Scope

GradeMate aims to develop a system that automates the grading of handwritten C++ programming quizzes. The core functionality involves handwriting recognition, allowing the conversion of handwritten C++ code into a digital format. Once digitized, the system evaluates the student's code by comparing it against a predefined rubric using a combination of structural (AST) and textual (Levenshtein) similarity techniques to ensure fair and consistent grading.

It also supports batch processing, enabling multiple quizzes to be graded simultaneously, a feature especially beneficial for larger class sizes.

Another key feature of GradeMate is plagiarism detection, which helps maintain academic integrity by identifying similarities between student submissions. The scope of the application is focused on foundational C++ programming concepts, excluding advanced topics like pointers and arrays. GradeMate primarily targets the automation of handwritten code scanning, grading, plagiarism detection, and cloud-based result management. The end goal is to deliver a functional and scalable prototype that addresses the growing need for efficient and intelligent grading systems in programming education.

1.3 Modules

The proposed GradeMate application will be divided into several key modules, each responsible for handling specific functionalities. Below is an outline of the main modules and their respective features:

1.3.1 Collect Dataset

The user-friendly features offered will be focused on streamlining and improving the efficiency of the handwritten quiz collection, scanning, and digitization processes. These are some of this module's primary features:

1. OCR Conversion and Quiz Scanning
2. Template Matching
3. Text Cleaning and Formatting

1.3.2 Model training

The user will be able to utilize tools that help them optimize the training process and improve the AI model. Making sure the AI model will generate accurate quiz responses and understand the C++ programming language is the main objective.

1. Fine-tune GPT-Neo Model
2. Dataset Preprocessing and Cleaning
3. Training the Model with C++ solutions

1.3.3 Quiz Evaluation

This module handles the automated evaluation of student quizzes by comparing them with a predefined teacher rubric. The system uses a combination of Levenshtein Distance (for textual similarity) and Abstract Syntax Tree (AST) comparison (for structural similarity). These techniques allow the system to account for multiple valid variations in student responses while ensuring logical and syntactic correctness. Features included are:

1. Code Comparison using Levenshtein and AST
2. Solution Customization
3. Solution Validation

1.3.4 Automated Grading

This module is essential for the automatic grading of student quizzes and for identifying cases of plagiarism. features provided to the user include:

1. Automated Grading System
2. Customizable Grading Rubrics
3. Plagiarism Detection

1.4 User Classes and Characteristics

With the primary goal of automating the grading of handwritten quizzes, the GradeMate system will serve various user classes within the educational system. The following user classes have been acknowledged along with the appropriate characteristics that they possess:

User class	Description
Instructor	Instructors will define grading criteria, approve rubrics, and oversee the grading process. The system will be used by them to make quizzes and track the completion of grading. When results from automated grading require manual confirmation, instructors may also come in. They will primarily use the system through a mobile application dashboard to oversee the process and analyze the outcomes.
Student	Students will also be the users of the system who will be submitting handwritten quizzes. These handwritten entries will be digitally converted by GradeMate, which will then automatically grade them. Multiple quizzes will be supported by the system, which is planned to be used by students using mobile applications. Additionally, students may interact with the system and get evaluations on their work.
Teaching Assistant (TA)	Teaching Assistants will be responsible for managing quiz submissions, reviewing the automated grading, and making any necessary manual adjustments. TAs will be the users of the system, often interacting with it to ensure grading accuracy and fairness. They may also generate quizzes using predefined templates and help configure grading rubrics.

Chapter 2

Project Requirements

2.1 Use-case Diagram

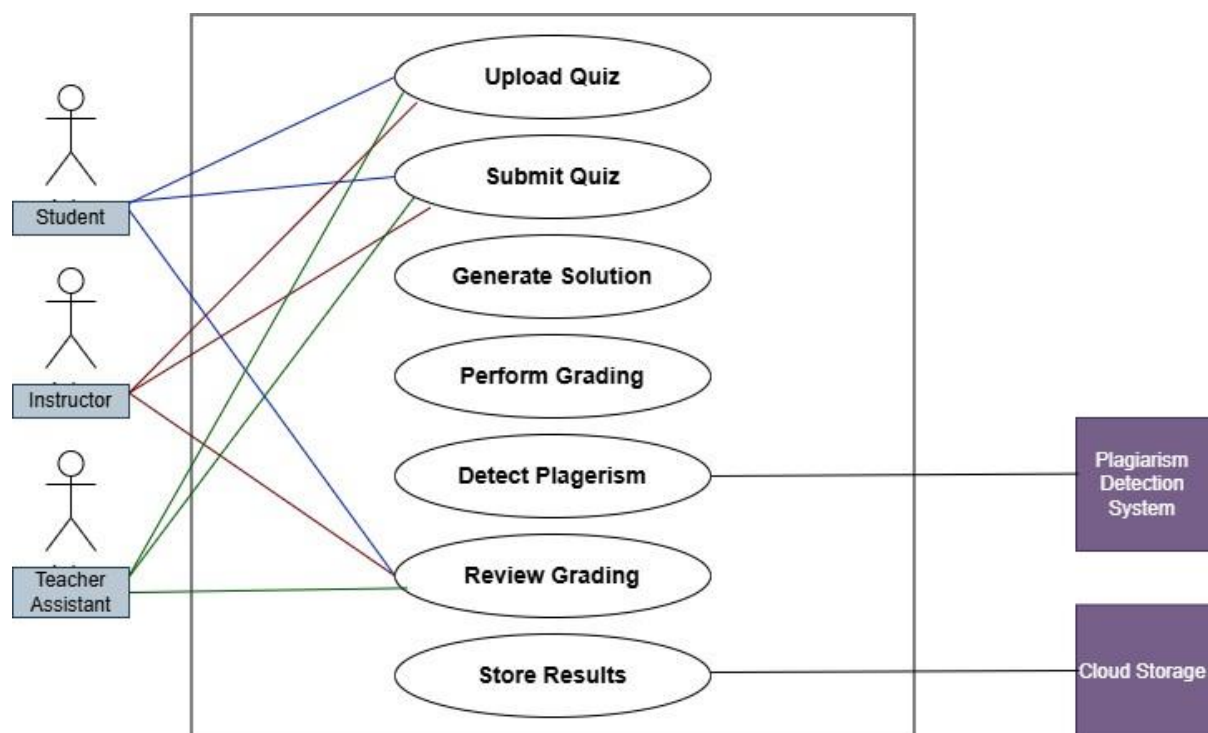


Figure 2.1: Use Case Diagram

Detailed Use Cases

ID	#1
Name	Manage Handwritten Quizzes
Short Description	Teachers can upload handwritten quizzes, define rubrics, and ensure quizzes are ready for automatic grading.
Goal	To allow teachers to securely upload quizzes and ensure they are processed and ready for grading by the system.
Preconditions	• Teacher must be logged into the system. • Teacher has quizzes in a suitable handwritten format.
Success End Condition	Handwritten quizzes are successfully uploaded and stored in the system, ready for grading.
Failure End Condition	Quizzes are not uploaded or fail to meet format requirements.
Stakeholder	Teacher
Trigger	Teacher logs in and uploads handwritten quizzes to the system.
Normal Flow	1. Teacher logs into the system. 2. Teacher navigates to the quiz upload section. 3. Teacher selects the handwritten quizzes to upload. 4. System validates the file format and uploads it to cloud storage. 5. Teacher defines rubrics and grading criteria.
Alternative Flows	• Teacher uploads a quiz with an unsupported format, and the system prompts to re-upload in the correct format. • Quiz upload is interrupted, and the system allows retrying.
Includes	Login, File Validation
Frequency of Use	Approximately 5-10 times per semester per teacher.
Constraints and Special Requirements	• Handwritten quizzes must be scanned in PDF or image formats. • Cloud storage integration must be secure and reliable.
Assumptions	• Teachers have access to scanning tools for uploading handwritten quizzes. • Internet connectivity is available during the upload process.
Notes and Issues	• Teachers may require training on supported formats and the upload process. • Large quiz files may face upload delays.

Table 2.1: Detailed Use Case: Manage Handwritten Quizzes

ID	#2
Name	Manage Grading Criteria
Short Description	Teachers can create, edit, and manage grading rubrics and criteria to evaluate student solutions.
Goal	To provide teachers with the ability to define grading rubrics and set the weightage for each operation in the grading process.
Preconditions	- Teacher must be logged into the system. - Teacher has access to the course and relevant assignments.
Success End Condition	Grading rubrics and criteria are successfully created or updated and are available for use in the grading process.
Failure End Condition	Grading rubrics and criteria are not created, or there is an error in defining the weightage or rubric content.
Stakeholder	Teacher
Trigger	Teacher logs into the system and selects the grading criteria management option.
Normal Flow	- Teacher logs into the system. - Teacher navigates to the grading criteria management section. - Teacher selects the option to create or edit a grading rubric. - Teacher defines grading criteria and assigns weightage to each operation or question. - Teacher saves the grading rubric and applies it to the relevant assignments.
Alternative Flows	- Teacher attempts to create a rubric but enters invalid weightage, prompting the system to flag an error. - Teacher decides to cancel changes, and the system reverts to the previous grading criteria without saving any changes.
Includes	Login, Rubric Creation, Rubric Editing
Frequency of Use	Approximately 1-3 times per semester per teacher, depending on the course structure.
Constraints and Special Requirements	- Rubric weightage must sum up to 100%. - System should support flexible rubric formats for different types of assessments (e.g., assignments, quizzes, projects).
Assumptions	- Teacher has prior knowledge of how to create grading rubrics. - Teachers have an understanding of the assessment types for which they are creating rubrics.
Notes and Issues	- Teachers may need training on the system's rubric management tools. - Incorrect weightage assignments can lead to miscalculation of grades.

Table 2.2: Detailed Use Case: Manage Grading Criteria

ID	#3
Name	View Graded Quizzes
Short Description	Students can view their graded quizzes, including scores and feedback from the teacher.
Goal	To allow students to review their graded quizzes, check their scores, and understand feedback.
Preconditions	• Student must be logged into the system. • Quiz grades and feedback must be available in the system.
Success End Condition	Student successfully views their graded quizzes, including detailed feedback and scores.
Failure End Condition	Student is unable to view graded quizzes due to missing grades or system errors.
Stakeholder	Student
Trigger	Student logs into the system and selects the option to view graded quizzes.
Normal Flow	1. Student logs into the system. 2. Student navigates to the "Graded Quizzes" section. 3. Student selects a specific quiz to view its score and feedback. 4. System displays the quiz score along with any detailed feedback from the teacher. 5. Student can review the quiz content, scores, and feedback.
Alternative Flows	• Student tries to view a quiz, but the grade is unavailable, and the system notifies them that the grading is in progress. • Student selects an incorrect quiz or attempts to view a quiz that hasn't been graded yet, leading to a prompt for retrying.
Includes	Login, Viewing Grades, Viewing Feedback
Frequency of Use	Approximately 1-5 times per semester per student, depending on the number of quizzes.
Constraints and Special Requirements	• Grades and feedback must be input by the teacher before students can view them. • The system should handle high volumes of traffic during grading periods.
Assumptions	• Students have access to the internet to view their graded quizzes. • Students are aware of where to find their graded quizzes in the system.
Notes and Issues	• System load may affect performance during peak times when grades are released. • Students may need guidance on how to interpret feedback and grades.

Table 2.3: Detailed Use Case: View Graded Quizzes

ID	#4
Name	Plagiarism Detection
Short Description	The system detects instances of plagiarism in students' submitted solutions to ensure academic integrity.
Goal	To ensure academic integrity by identifying plagiarism in student submissions.
Preconditions	- Teacher or student is logged into the system. - Student submissions are uploaded to the system.
Success End Condition	Plagiarism is detected, and a detailed report is generated, highlighting the instances of plagiarism.
Failure End Condition	No plagiarism is detected, or there is a system failure in generating the report.
Stakeholder	Teacher, Student, Any User
Trigger	Teacher or student uploads a submission for plagiarism checking.
Normal Flow	- Teacher or student logs into the system. - Teacher or student uploads a student's solution for plagiarism detection. - System scans the submitted solution against a database of known sources. - System identifies similarities and generates a plagiarism report. - Report is provided to the teacher or student, highlighting the plagiarized content and the sources.
Alternative Flows	- System fails to detect plagiarism due to a database error or missing resources. - User uploads an unsupported file type, and the system prompts the user to upload a valid file.
Includes	Login, File Upload, Report Generation
Frequency of Use	Varies depending on the number of student submissions, but typically once per assignment per student.
Constraints and Special Requirements	- The plagiarism detection system must have access to a large and up-to-date database of sources. - The system should be able to handle different file formats (e.g., text, PDF, code). - Report generation should be fast and detailed enough for teachers to take action.
Assumptions	- Students submit their solutions through the system. - Teachers are familiar with how to interpret the plagiarism report.
Notes and Issues	- False positives or false negatives may occur in plagiarism detection, requiring manual review. - Some students may use paraphrasing techniques to avoid detection.

Table 2.4: Detailed Use Case: Plagiarism Detection

ID	#5
Name	Dataset Preprocessing and Cleaning
Short Description	Cleans and preprocesses raw data (e.g., handwritten C++ solutions, student submissions) to ensure proper formatting for model training.
Goal	Prepare data for effective model training by eliminating errors and ensuring proper formatting.
Preconditions	• User is logged in. • Raw datasets are available and readable. • Dataset requires cleaning (e.g., text extraction, noise removal).
Success End Condition	Dataset is cleaned, formatted, and ready for model training.
Failure End Condition	Dataset cleaning fails, preventing further training.
Stakeholder	Teacher, System Administrator
Trigger	User selects dataset cleaning option.
Normal Flow	1. User logs into the system. 2. Uploads raw datasets (e.g., handwritten C++ solutions). 3. System preprocesses the data (e.g., noise removal, feature extraction). 4. User reviews and confirms cleaned data. 5. System stores the cleaned dataset.
Alternative Flows	• System encounters errors and prompts for corrections. • Poor data quality leads to inadequate results.
Includes	Data Review, Preprocessing, Error Handling
Frequency of Use	Regularly, with new datasets for training.
Constraints	• Raw data must be readable. • Preprocessing steps adapt to dataset types.
Assumptions	• Raw data is in a processable format. • User understands preprocessing importance.
Notes and Issues	• Processing time varies by dataset size and complexity.

Table 2.5: Detailed Use Case: Dataset Preprocessing and Cleaning

2.2 Functional Requirements

2.2.1 Module 1: Collecting Dataset

Following are the requirements for module 1:

1. FR1: Teachers and teaching assistants (TAs) will be able to upload handwritten quiz submissions in specified formats (JPEG and PNG) using the system.
2. FR2: The system shall automatically extract text from handwritten quiz submissions using Optical Character Recognition (OCR) tools
3. FR3: To increase model accuracy during training, the system should enable integration with external datasets (IAM, MNIST, InftyCBD-2 etc.).
4. FR4: The system will keep track of every quiz submission in the database, associating it to the student's individual ID and quiz number.

2.2.2 Module 2: Model Training

Following are the requirements for module 2:

1. FR1: The technology will enable the use of hybrid datasets, like EMNIST and quizzes submitted by students, to train machine learning models, including neural networks.
2. FR2: The system must enable saving of the trained model for further use and upgrades.
3. FR3: To enhance performance, the system should allow users (teachers or system administrators) to modify important model parameters including learning rate, batch size, and epochs.
4. FR4: Following each model training session, the system is required to produce performance metrics, such as accuracy, recall, and F1-score.
5. FR5: The system shall allow re-training of the model using new datasets or student submissions to ensure the model remains accurate and up-to-date.

2.2.3 Module 3: Quiz Evaluation

Following are the requirements for module 3:

1. FR1: Teachers can define custom grading rubrics by assigning weightages to logic similarity (AST) and text similarity (Levenshtein).
2. FR2: The system shall compare the student's handwritten solution with the teacher's reference solution using Levenshtein distance and AST-based structural comparison.
3. FR3: The system shall compute a final score and assign a grade based on the combined similarity scores and the provided rubric.
4. FR4: The system shall allow teachers to manually override or adjust grades if they deem the automated evaluation insufficient or inaccurate.
5. FR5: The system shall highlight textual and structural differences between the teacher's and student's code to provide visual insight into grading decisions.

2.2.4 Module 4: Automated Grading

Following are the requirements for module 4:

1. FR1: Using the established rubrics, the system will automatically evaluate each quiz submission by comparing the responses provided by the students with the generated solutions.
2. FR2: The system will enable teachers to grade assignments manually, giving them the ability to modify or overturn grades that are generated automatically.
3. FR3: The system will link each grade to the appropriate quiz submission by storing student grades in the database.

2.3 Non-Functional Requirements

2.3.1 Reliability

1. REL-1: Under typical operating conditions, the GradeMate system must have a mean time between failures (MTBF) of at least a thousand hours.
2. REL-2: With a high accuracy rate, the system will identify grading errors or mismatches in student submissions.
3. REL-3: To prevent data loss in the case of a system failure, the system must enable teachers to access student information and grading history from the most recent checkpoint.

4. REL-4: Automated grading of quizzes shall have a less failure rate. Error logs will be maintained to record and notify any system failures

2.3.2 Usability

1. USE-1: The GradeMate system's user interface (UI) shall be intuitive and easy to use, for new users (teachers, TAs, and students).
2. USE-2: With just three clicks, the system should enable teachers to upload quizzes.
3. USE-3: Students will only need to click on their dashboard to view their graded quizzes.
4. USE-4: The system will offer error alerts and help for typical user mistakes, including uploading the wrong file format for a quiz.

2.3.3 Performance

1. PER-1: The system shall process the upload of a handwritten quiz in under 60 seconds with faster internet connection.
2. PER-2: For quizzes with 20 submissions or more, automatically graded results must be accessible in 10 minutes.
3. PER-3: During peak hours, the system can accommodate up to 100 students and 50 teachers using it concurrently without experiencing any performance reduction.
4. PER-4: For the best possible user experience, the GradeMate system must load each page or activity in less than two seconds.

2.3.4 Security

1. SEC-1: The system must guarantee that encryption standards are applied to all quiz data, student submissions, and grade.
2. SEC-2: The users should add credentials to enforce user authentication for instructors and teaching assistants.
3. SEC-3: To guard against unauthorized access or modifications, the system must record and keep track of every activity made by users (teachers, TAs, and students). These logs must be kept for a period of 90 days.

4. SEC-4: Access control will be put into place to make sure that only individuals with permission can read and edit tests, grades, or student information. After three unsuccessful unauthorized access attempts, the user account will be locked and alerts will be sent out.

2.4 Domain Model

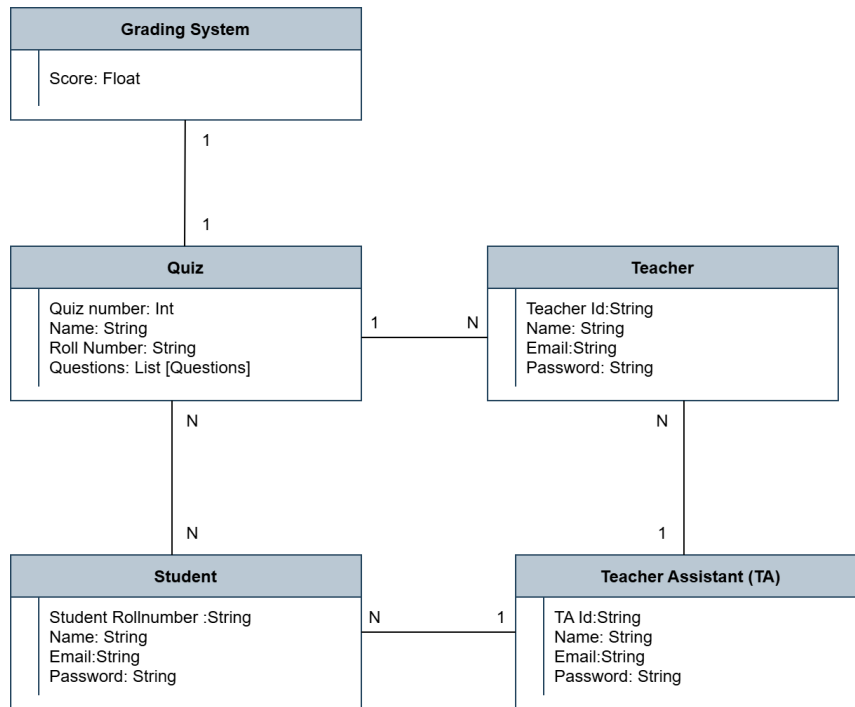


Figure 2.2: Domain Model

Chapter 3

System Overview

3.1 Architectural Design

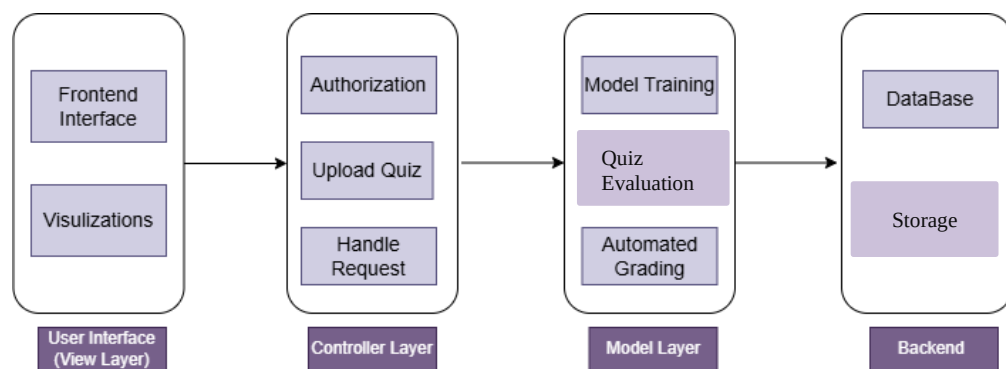


Figure 3.1: The architectural design of the GradeMate system

3.2 Design Models

Activity Diagrams

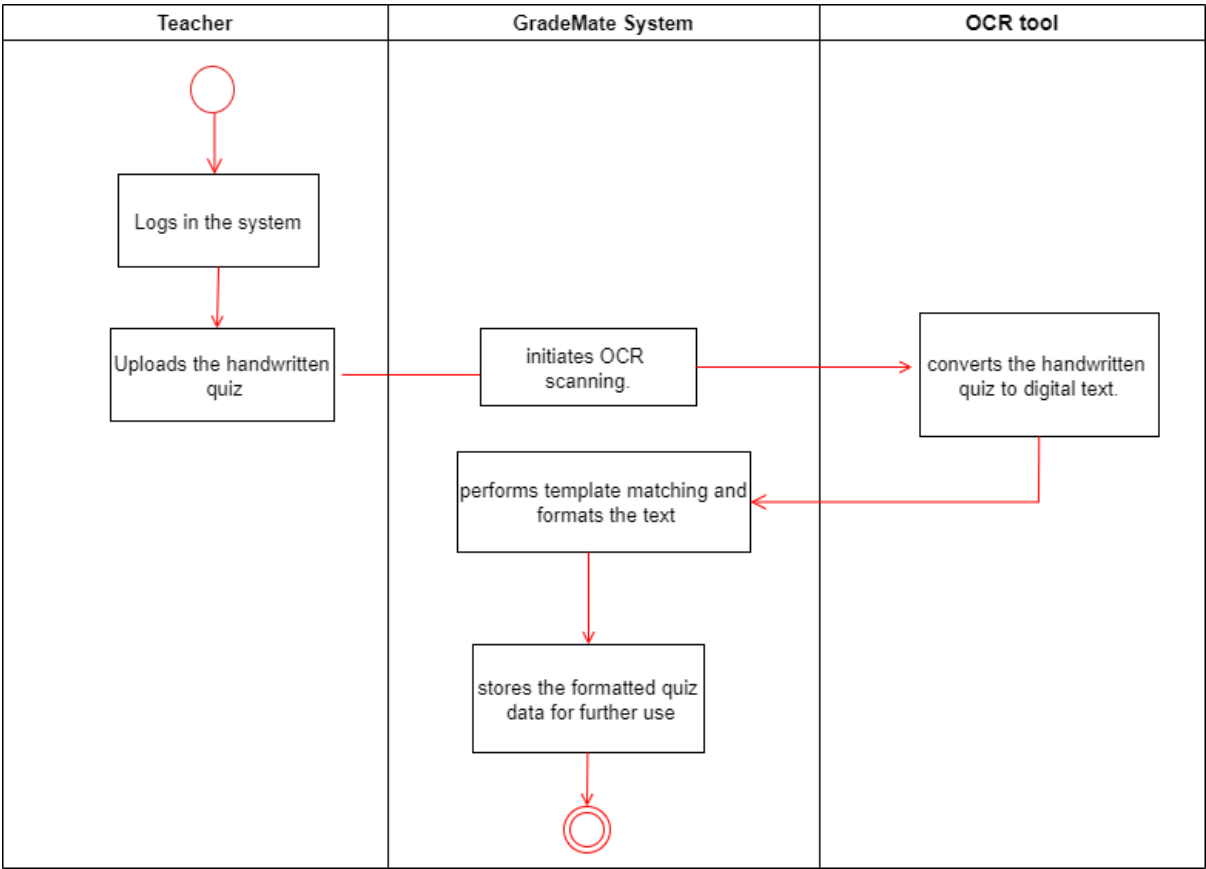


Figure 3.2: Data collection Module

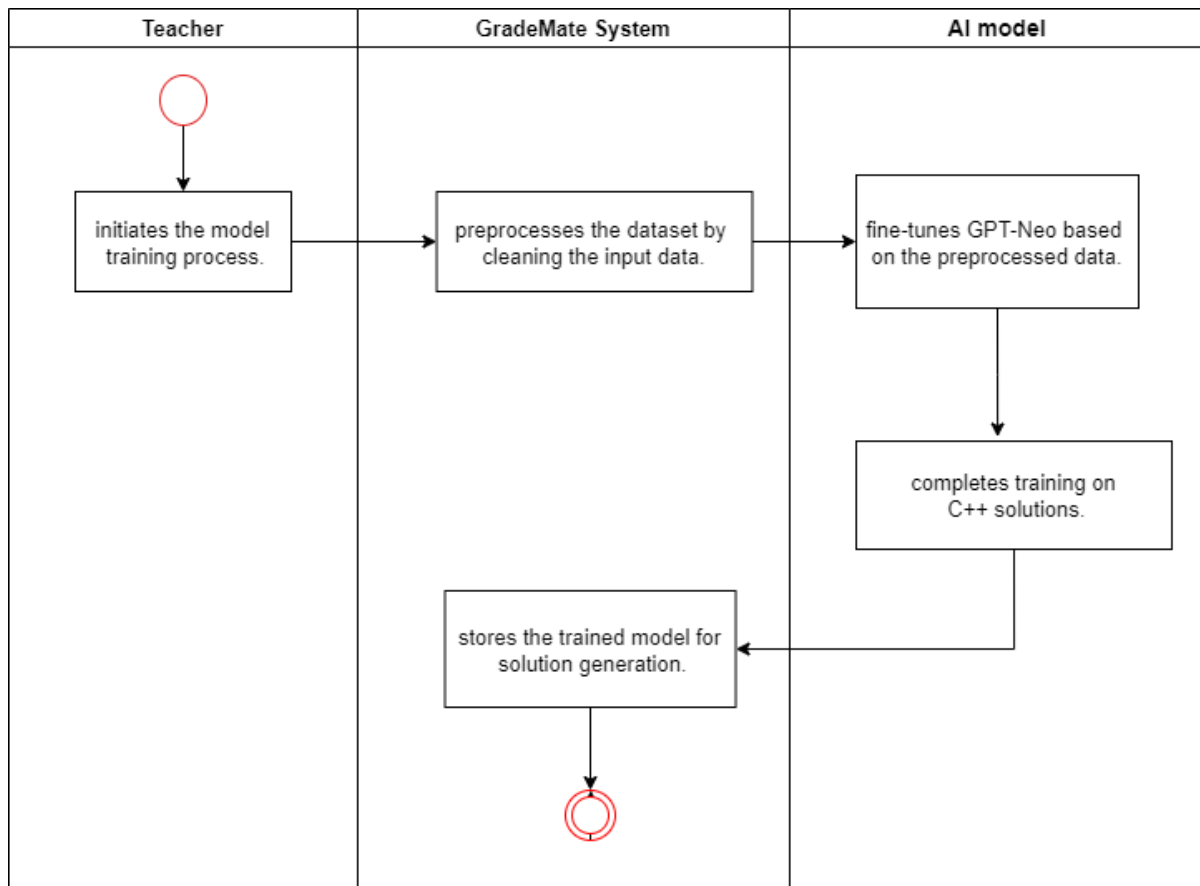


Figure 3.3: Model Training Module

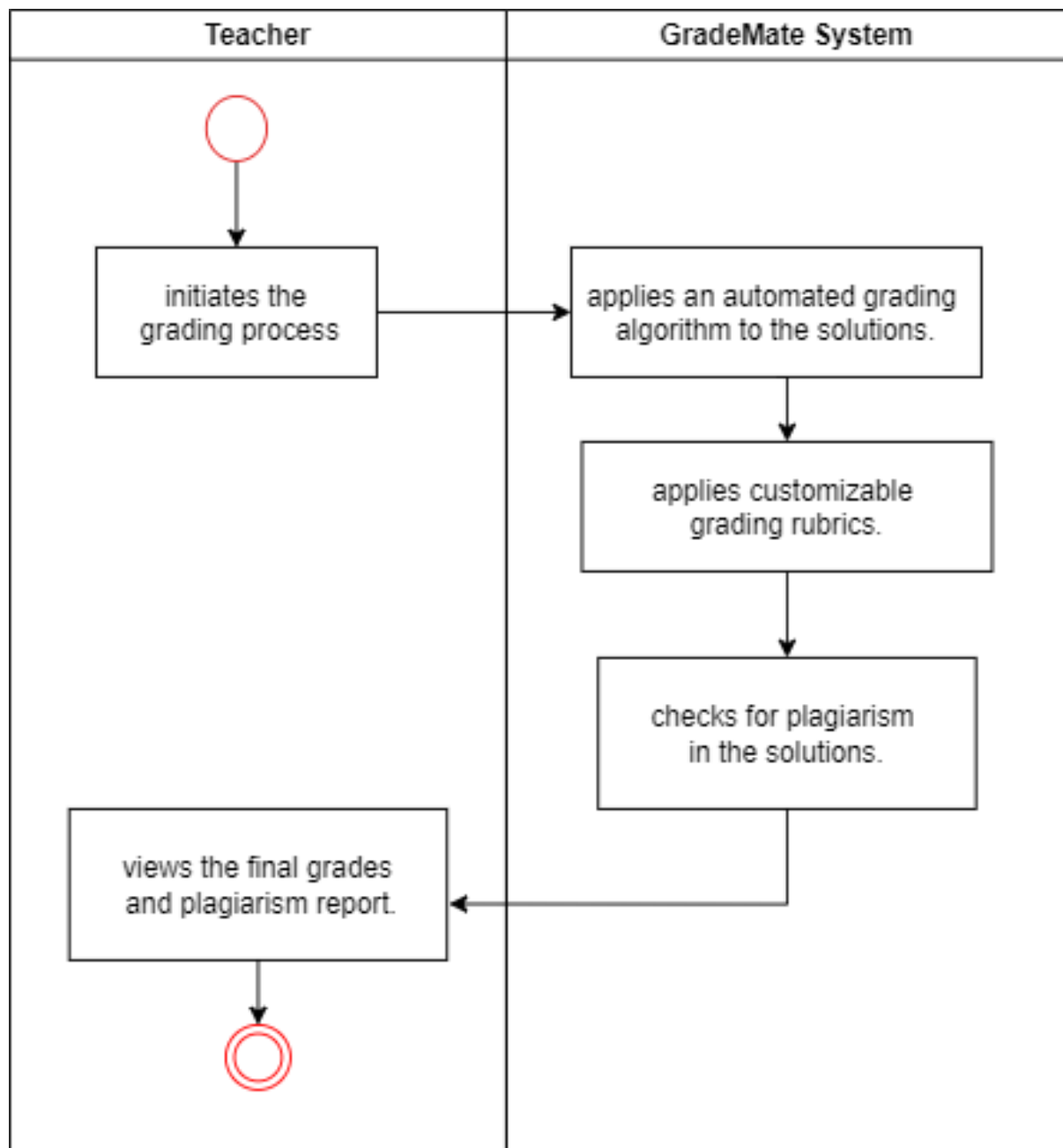


Figure 3.4: Automated Grading Module

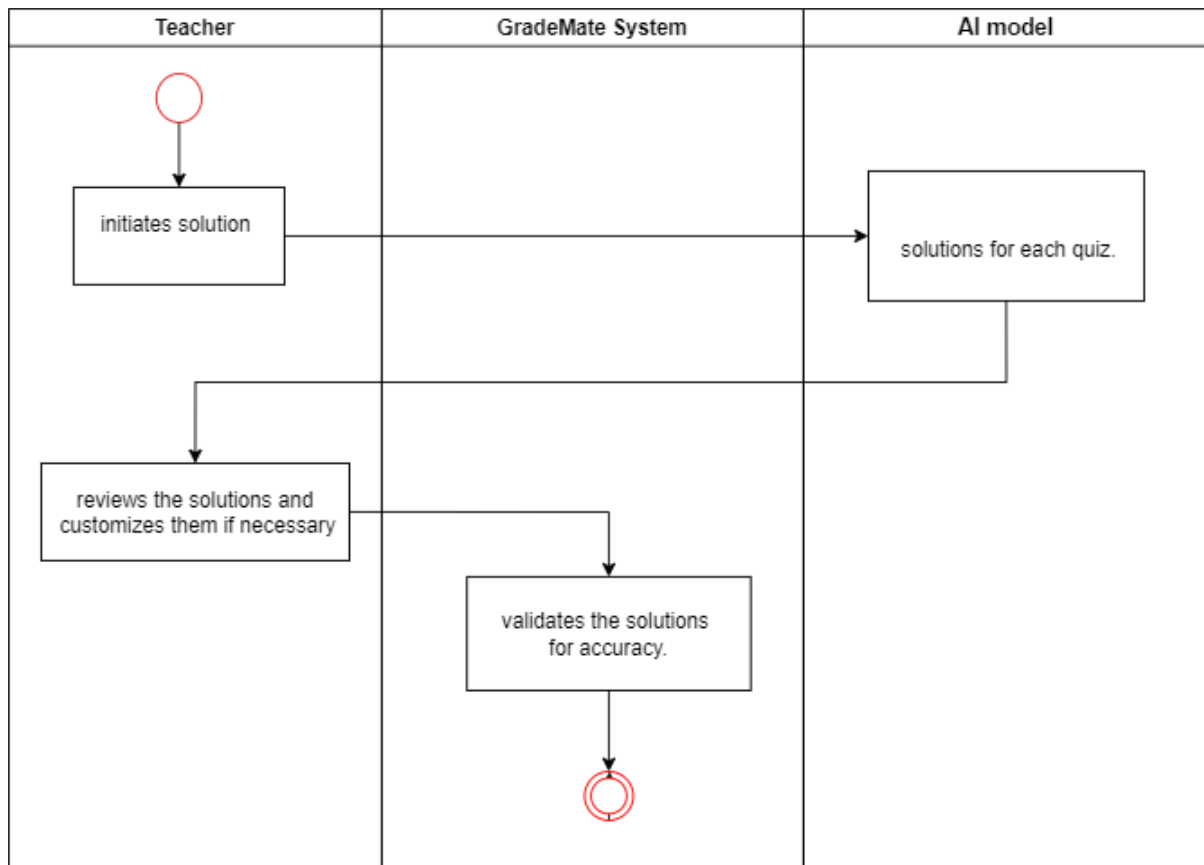


Figure 3.5: Quiz Evaluation Module

Sequence Diagrams

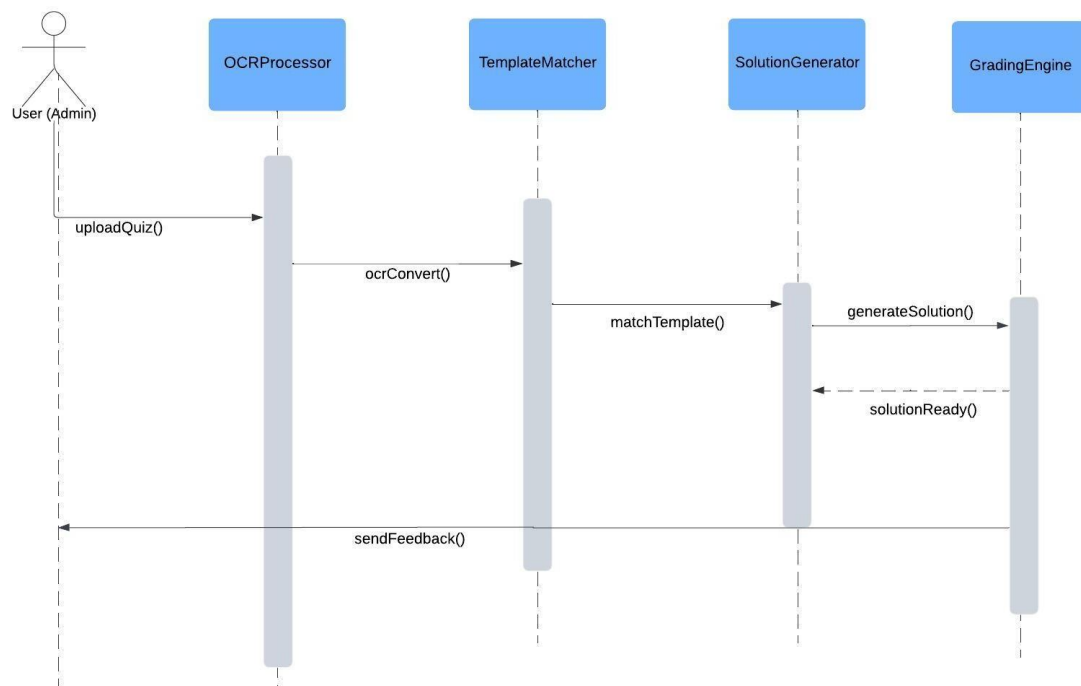


Figure 3.6: OCR Conversion and Quiz Scanning

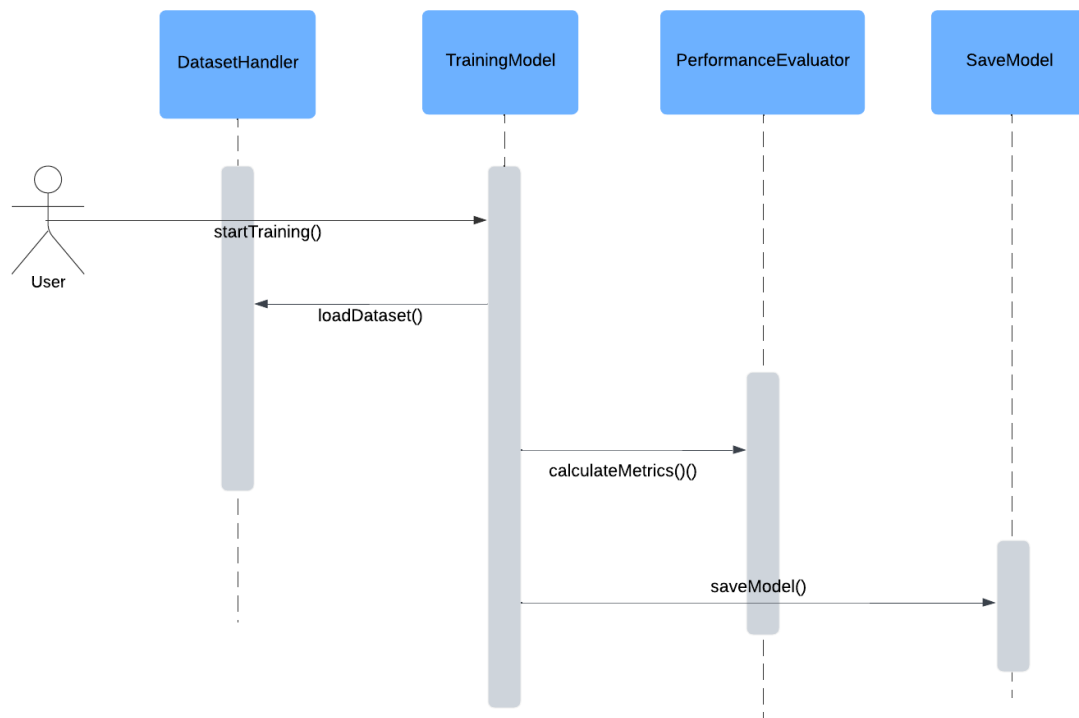


Figure 3.7: Model Training

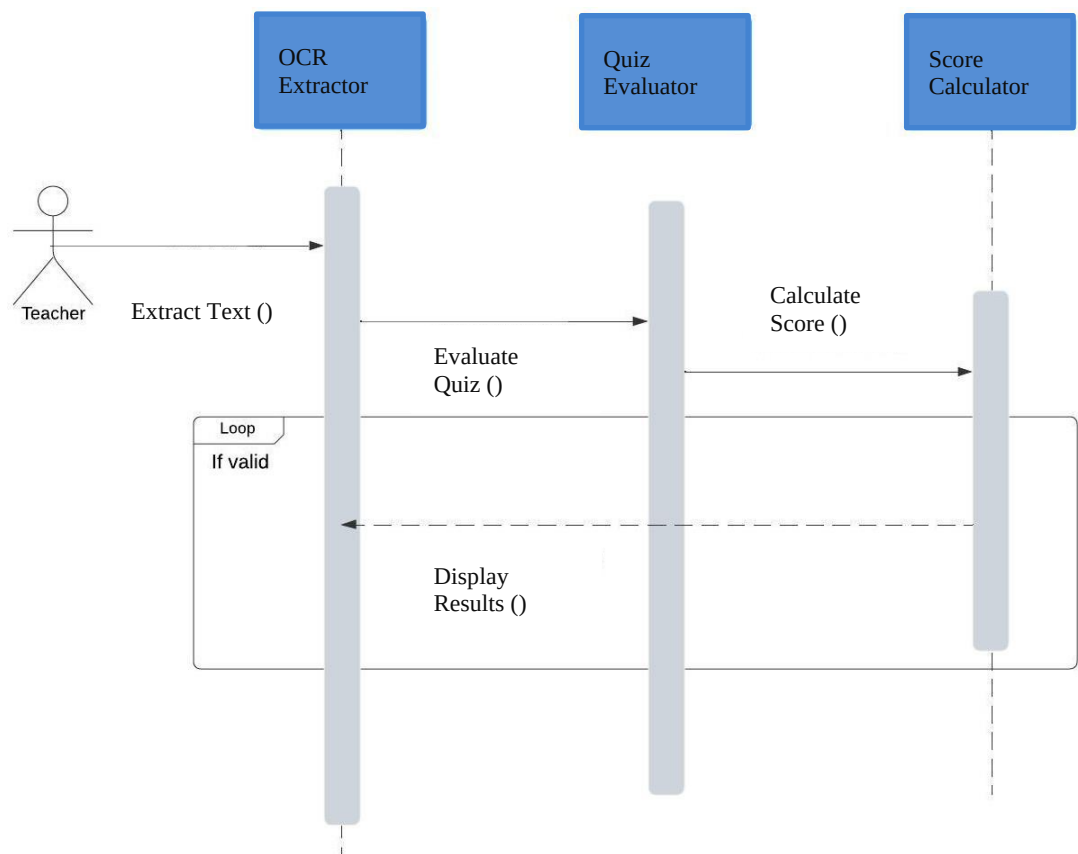


Figure 3.8: Quiz Evaluation

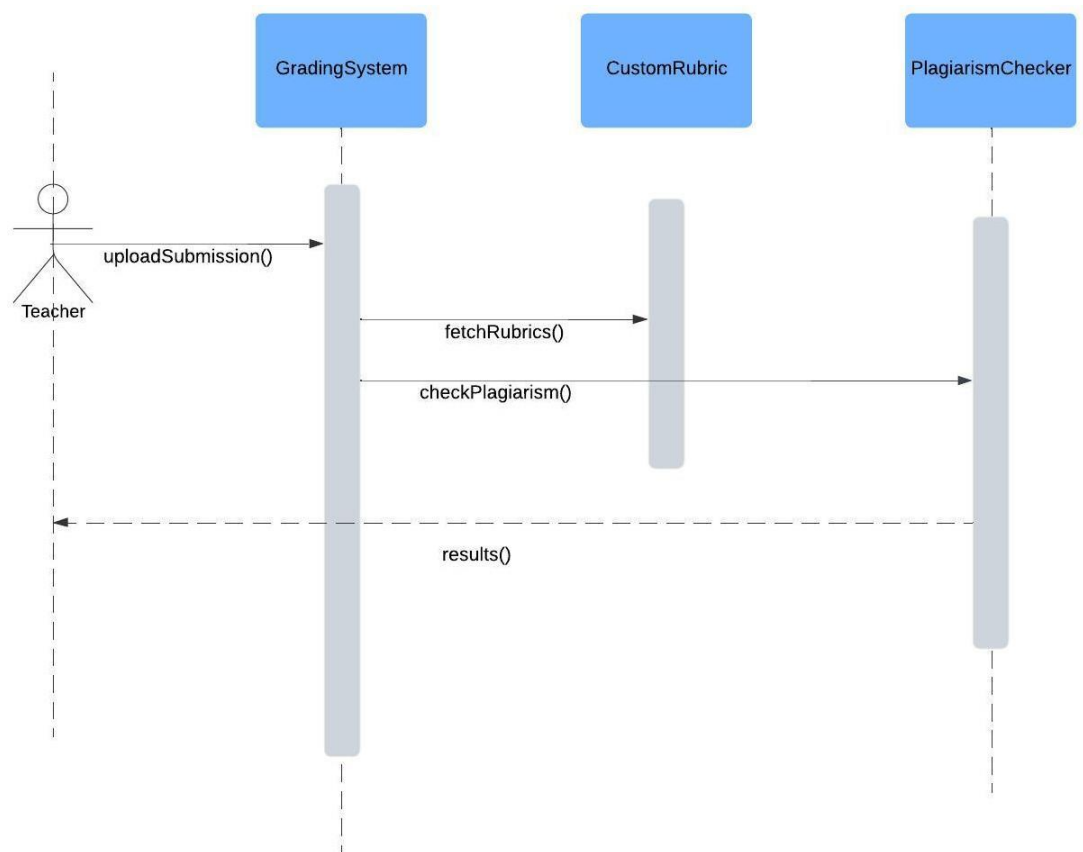


Figure 3.9: Automated Grading

State Transition Diagram

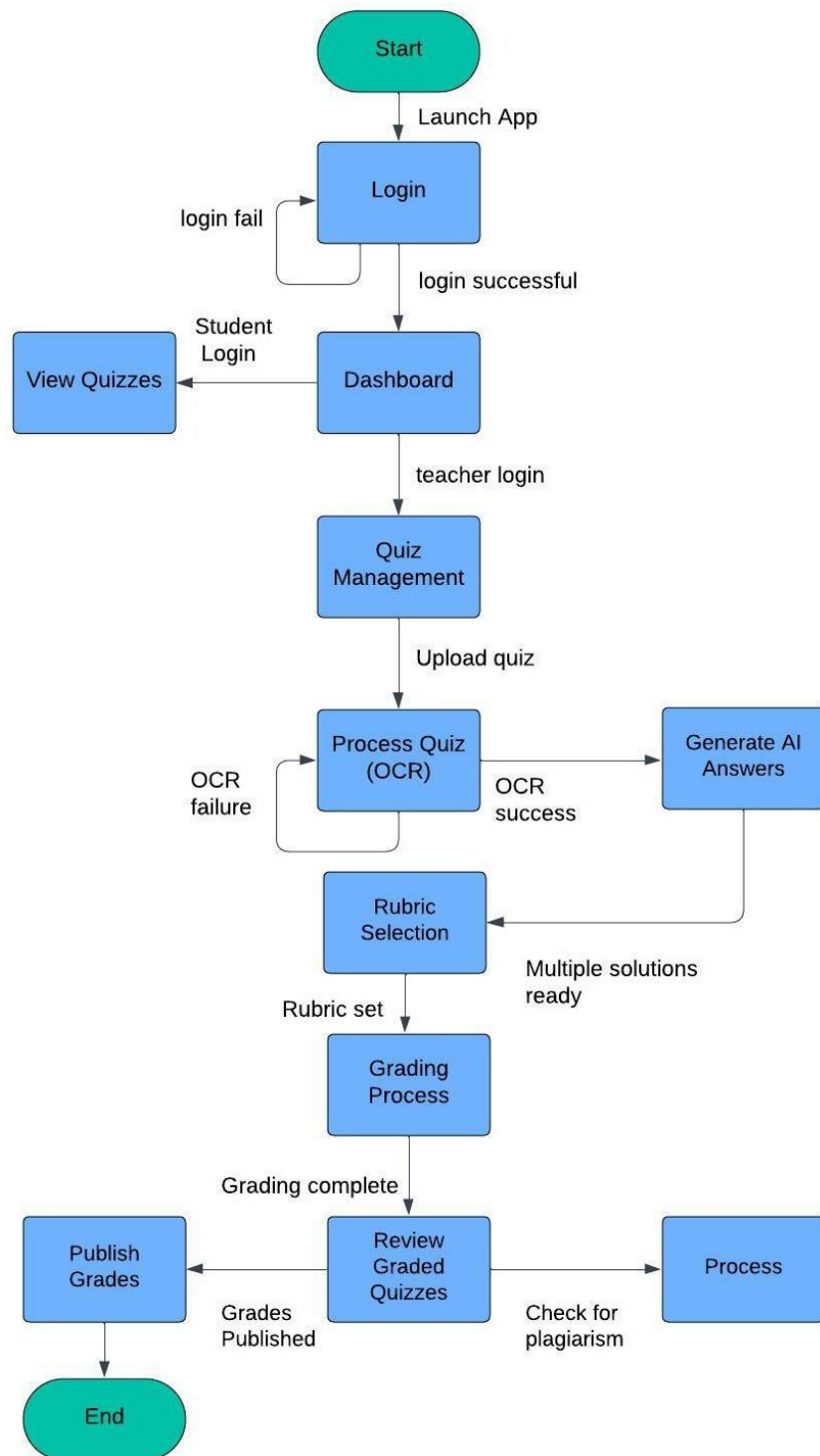


Figure 3.10: State Transition Diagram

Class Diagram

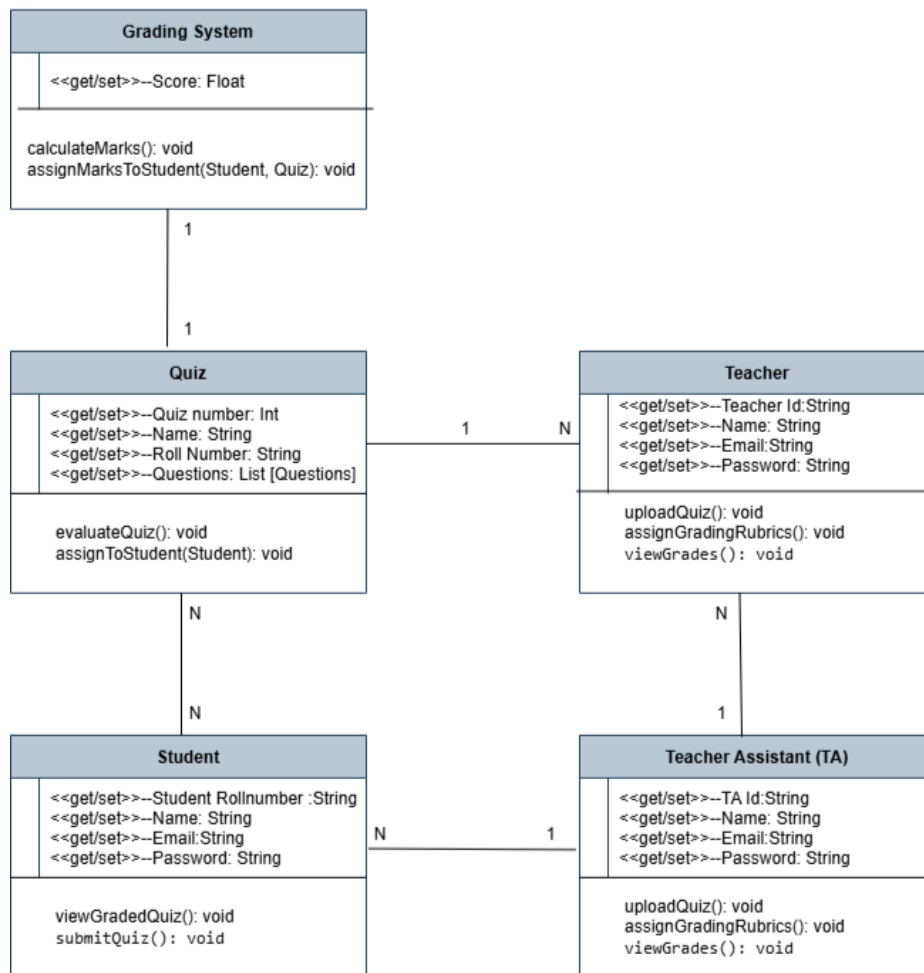


Figure 3.11: Class Diagram

3.3 Data Design

3.3.1 Transformation of Information Domain into Data Structures

Structured, semi-structured, and unstructured data (such as handwritten quizzes) are all handled by the system. The process of converting information into data structures is as follows:

1. Quizzes: The primary data entity is quizzes, each of which has different attributes: . It is

linked to student submissions and grades.

2. Users: Multiple user roles, such as students, teachers, and teaching assistants (TAs), are supported by the system. A user's name, email address, user type, credentials, and user ID are among their attributes.
3. HandWritten Quiz Data: To extract text from student quiz images, OCR (Optical Character Recognition) is used. These images are stored and linked to the corresponding quizzes and students.
4. Grades: Each quiz attempt is graded, and the grades are stored in structured format with attributes like marks, and quizNumber.

3.3.2 Storage of Major Data Entities

1. Relational Databases: Relational databases like MySQL are used to store structured data, such as user information, quiz results, and grades. The following tables are widely used:
 - Users Table: Stores user data, differentiating between students, teachers, and TAs.
 - Quizzes Table: Holds details about every quiz.
 - Grades Table: Using foreign keys to link students and quizzes, this table holds the grades for each attempt at a quiz.
2. Handwritten Quiz Images: Handwritten quizzes are kept in cloud storage systems. These files are linked to quizzes and students in the relational database.
3. Plagiarism Data: The system stores data from plagiarism detection processes. The Plagiarism Results Table holds attributes such as plagiarismScore, and comparisonDetails

3.3.3 Processing of Major Data Entities

- OCR Processing: OCR tools are used to extract text, which is then compared against a set of model solutions.
- Automated Grading: The system compares students' responses to model solutions using a combination of machine learning and pre-established grading rubrics. Accuracy, completeness, and compliance to the question structure are taken into consideration while awarding grades.
- Plagiarism Detection: To identify possible plagiarism, programs such as MOSS will compare student submissions to both previous and current data.

3.3.4 Organization of Data Entities

1. User-Based Data Organization: Data is structured based on the role of the user:
2. Quiz Data Organization: Each quiz is linked to its respective students, with relationships.

3.3.5 Databases and Data Storage Items

1. Relational Database (MySQL) Stores structured data such as quiz information, grades, user details, and plagiarism results.
2. Cloud Storage Stores unstructured data such as images of handwritten quizzes

Chapter 4

4.1 Implementation and Testing

4.1.1 Functionality, Context, and Design

GradeMate is an innovative solution developed to automate the grading of handwritten C++ programming quizzes. This system leverages Optical Character Recognition (OCR), machine learning, and AI-driven text similarity algorithms to streamline the evaluation process while ensuring accuracy and reliability.

4.1.1.1 Functionality

The primary functionality of GradeMate includes:

- (a) **OCR-based Text Extraction:** Handwritten quizzes are scanned and converted into editable digital text using EasyOCR, ensuring compatibility with subsequent processing stages.
- (b) **Rubric-Based Quiz Evaluation:** The extracted student code is compared against the teacher-provided reference solution using a grading rubric defined by the teacher. This rubric includes customizable weightages for logic accuracy (using AST comparison) and textual similarity (using Levenshtein distance).
- (c) **Automated Grading:** Student solutions are evaluated against generated solutions based on predefined grading rubrics, which assess criteria like correctness, logic, and code style.
- (d) **Plagiarism Detection:** A specialized tool (MOSS) is integrated to identify any instances of copied solutions, ensuring academic integrity.

4.1.1.2 Context

GradeMate was designed in response to the challenges faced in manually grading hand-written quizzes, such as inconsistency, inefficiency, and limited scalability. It addresses the following key issues:

- Time-consuming manual grading processes.
- Subjectivity and inconsistency in evaluations.
- Difficulty in comparing alternate solutions provided by students.
- Academic dishonesty in the form of plagiarism.

The system is tailored to meet the requirements of instructors, students, and teaching assistants in academic institutions, focusing on foundational C++ programming concepts.

4.1.1.3 Design

The design of GradeMate revolves around modularity and scalability. It is organized into the following core modules:

- (a) **Data Collection:** Includes OCR conversion, text cleaning, and formatting to prepare student submissions for analysis.
- (b) **Model Training:** Facilitates fine-tuning of the GPT-Neo model using hybrid datasets, including student submissions and publicly available handwriting datasets such as EMNIST.
- (c) **Rubric-Based Evaluation:** Instead of relying on automated solution generation, the system evaluates the student's converted code against a teacher-provided reference solution. It uses a hybrid evaluation mechanism combining Abstract Syntax Tree (AST) comparison and Levenshtein distance to analyze both logic and textual similarity.
- (d) **Automated Grading:** Assigns scores based on rubric evaluation and stores grades in a relational database, ensuring traceability.

The system employs a layered architecture with a cloud-based backend for data storage and processing, ensuring secure access and scalability for large class sizes. The frontend is designed to be user-friendly, minimizing the learning curve for educators and students alike.

4.1.1.4 Background Information

The project builds on established technologies and frameworks:

- **OCR Conversion:** EasyOCR library for accurate text extraction from handwritten quizzes.
- **Language Model:** GPT-Neo for solution generation, fine-tuned to understand C++ syntax and logic.
- **Plagiarism Detection:** MOSS for identifying duplicate solutions across submissions.
- **Database Management:** MySQL for structured storage of user details, quizzes, and grades, integrated with a cloud-based file storage system for unstructured data.

By integrating these technologies, GradeMate ensures a comprehensive and robust system for automating the grading process while maintaining academic fairness and efficiency.

4.2 Algorithm Design

The implementation of GradeMate relies on a combination of algorithms designed to automate the evaluation of handwritten C++ programming quizzes. Each core module incorporates a specialized algorithm, ensuring accuracy, efficiency, and scalability. Below are the key algorithms used in the project, along with their respective pseudocodes.

This module utilizes the EasyOCR library for recognizing handwritten text and converting it to machine-readable format. The pseudocode for extracting text from handwritten images is shown below:

```
1 Algorithm: OCR-based Text Extraction
2 Input: Image of the handwritten quiz (I)
3 Output: Extracted text in string format (T)
4
5 1. Preprocess the input image I (grayscale conversion, noise removal)
6 2. Apply EasyOCR to detect text regions
7 3. Extract text lines and concatenate them into a single string T
8 4. Return T
```

Figure 4.1: Pseudocode for OCR-based Text Extraction.

A hybrid rubric-based evaluation mechanism is used to compare the student's handwritten solution with the reference solution provided by the teacher. This method incorporates both structural (AST) and textual (Levenshtein) similarity to account for variations in code logic and writing style. The pseudocode for rubric-based quiz evaluation is shown below

```

1 Algorithm: Alternate Solution Generation
2 Input: Extracted question text (Q), fine-tuned GPT-Neo model (M)
3 Output: Set of valid alternate solutions (S)
4
5 1. Preprocess Q to ensure syntax compatibility
6 2. Pass Q as input to the GPT-Neo model M
7 3. Collect generated solutions S
8 4. Filter solutions to remove syntactically or logically incorrect outputs
9 5. Return S

```

Figure 4.2: Pseudocode for rubric-based quiz evaluation

This module compares the student's solution against alternate solutions using predefined rubrics and assigns a score. The pseudocode for the Automatic Grading is shown below:

```

1 Algorithm: Automated Grading
2 Input: Student solution (S_s), alternate solutions (S_a), grading rubric (R)
3 Output: Final score (Score)
4
5 1. Initialize Score  $\leftarrow 0$ 
6 2. For each criterion  $r \in R$ :
7   2.1 If S_s satisfies r compared to any s  $\in S_a$ :
8     2.1.1 Increment Score by r.weight
9 3. Return Score

```

Figure 4.3: Pseudocode for Automatic Grading.

This module employs MOSS to identify similar solutions across submissions, ensuring academic integrity. The pseudocode for the Plagiarism Detection is shown below:

```

1 Algorithm: Plagiarism Detection
2 Input: Set of student solutions (S)
3 Output: Plagiarism report (P)
4
5 1. Upload all solutions S to the MOSS system
6 2. Retrieve similarity scores and flagged pairs of solutions
7 3. Generate plagiarism report P based on flagged pairs
8 4. Return P

```

Figure 4.4: Pseudocode for Plagiarism Detection.

4.3 External APIs/SDKs

The following table describes the third-party APIs and SDKs used in the implementation of the project, along with their purpose and specific endpoints or functions used.

API and version	Description	Purpose of usage	API endpoint/function/class used
EasyOCR (latest release)	OCR library for handwritten text recognition	Digitization of quizzes and C++ code parsing	<code>easyocr.Reader()</code>
OpenAI GPT-Neo API	AI language model	Fine-tuning GPT-Neo for solution generation	<code>generate()</code>
Moss	Plagiarism detection for source code	Check for plagiarism in student quiz solutions	https://theory.stanford.edu/aiken/moss/

4.4 Testing Details

4.4.1 Unit Testing

Each unit test is designed to test a specific function or method independently from other components, helping to identify issues directly related to the functionality being tested.

Test Case ID	Test Objective	Preconditions	Steps	Test Data	Expected Result	Post Condition	Pass/Fail
TC001	Verify Upload Quiz functionality	User must be logged in	1. Navigate to Quiz Upload page. 2. Click "Choose File." 3. Select and upload a file.	File: quiz.pdf	Quiz uploaded successfully with a confirmation message.	User is directed to the OCR conversion page.	Pass

Table 4.1: Unit Test Case for quiz upload.

Test Case ID	Test Objective	Preconditions	Steps	Test Data	Expected Result	Post Condition	Pass/Fail
TC002	Validate OCR Conversion	A valid quiz file must be uploaded.	1. Upload handwritten quiz via "Upload Quiz." 2. Click on "Convert to Text."	File: handwritten-quiz.png	OCR text output displayed in the preview box.	User can edit the text before generating a solution.	Pass

Table 4.2: Unit Test Case for OCR conversion.

Test Case ID	Test Objective	Preconditions	Steps	Test Data	Expected Result	Post Condition	Pass/Fail
TC003	Verify Automated Solution Generation	OCR conversion must have completed.	1. Click "Generate Solution" on the converted text. 2. Wait for response.	OCR text: C++ code snippet	Automatically generated solution displayed in the solution box.	The solution is stored in the database for review.	Pass

Table 4.3: Unit Test Case for Quiz Evaluation

Test Case ID	Test Objective	Preconditions	Steps	Test Data	Expected Result	Post Condition	Pass/Fail
TC004	Verify Rubric-Based Quiz Evaluation	Teacher and student submissions must be uploaded	1. Upload teacher solution and student quiz. 2. Enter rubric weights. 3. Trigger grading module.	Teacher and student code snippets	System evaluates both solutions using AST + Levenshtein and assigns score.	Final score and grade are generated and displayed.	Pass

Table 4.4: Unit Test Case for model training.

Test Case ID	Test Objective	Preconditions	Steps	Test Data	Expected Result	Post Condition	Pass/Fail
TC005	Verify Custom Grading Guidelines	User must be logged in as teacher.	1. Navigate to grading section. 2. Set custom grading guidelines and rubrics.	Grading Guidelines: Correctness, Logic, Code Style	System saves the custom guidelines.	Guidelines are applied to quizzes.	Pass

Table 4.5: Unit Test Case for grading criteria.

Test Case ID	Test Objective	Preconditions	Steps	Test Data	Expected Result	Post Condition	Pass/Fail
TC006	Validate Text Similarity Check	A model-generated solution must exist.	1. Upload student handwritten solution. 2. System compares with model-generated solution.	Solution Text: Student's answer, Model's answer	System computes similarity score and shows result.	Similarity score is recorded for analysis.	Pass

Table 4.6: Unit Test Case for plagiarism detection.

Chapter 5

Conclusions and Future Work

5.1 Conclusion

GradeMate successfully addresses the long-standing challenge of manually grading hand-written C++ programming quizzes by introducing an automated, intelligent, and scalable solution. Through the integration of OCR-based text extraction, rule-based grading mechanisms, Levenshtein distance, AST (Abstract Syntax Tree) comparison, and plagiarism detection, the system streamlines the entire grading process while ensuring fairness, consistency, and efficiency. Despite limitations in developing a fully functional custom OCR model due to suboptimal results, the use of a tuned OCR engine with advanced techniques provided robust and reliable text recognition performance. The use of AST and Levenshtein proved pragmatic in light of cost, model compatibility, and complexity constraints.

Overall, GradeMate stands as a strong prototype system that demonstrates the potential of AI and automation in the education technology space, significantly reducing instructor workload and enabling faster feedback for students.

5.2 Future Work

- (a) **Improved Plagiarism Detection:** Expand the plagiarism detection feature with support for advanced techniques beyond MOSS, such as semantic code similarity and stylometry.
- (b) **Scalability and Deployment:** Transition the prototype into a fully deployed web-based SaaS platform capable of handling institution-wide usage with role-based access and analytics dashboards.
- (c) **Manual Human Correction Override:** Provide an option for teachers to review, edit, or override the automated grading if they believe the evaluation is incorrect,

biased, or incomplete. This ensures transparency and adds a human layer of judgment where needed.

Bibliography

- [1] A Kolyshkin and S Nazarovs. Stability of slowly diverging flows in shallow water. *Mathematical Modeling and Analysis*, 2007.